

第1章-3: Jetpack Compose 入门讲义 (小白专属)

讲师寄语: 各位刚接触Android开发的小伙伴，之前咱们写UI可能都用过XML布局，是不是经常对着一堆标签头晕？今天学的Jetpack Compose（简称Compose），就是Google帮咱们解放双手的"UI新工具"。它用纯Kotlin代码写UI，就像搭积木一样简单，咱们跟着例子敲代码，保证你一看就会！

一、先搞懂：Compose到底好在哪？（Compose优势）

咱们先不聊复杂概念，先说说Compose比老办法（XML）好在哪，帮你建立"为什么要学"的信心：

1. **少写代码，多做事情：**以前写一个按钮，XML里定义样式，Kotlin里写点击事件，来回切换文件。Compose用一段代码搞定，不用跨文件折腾。
2. **所见即所得，调试快：**修改代码后，预览窗口秒更新，不用等App重新运行，像写Word文档实时看到效果一样。
3. **逻辑和UI不分家：**数据变了UI自动更，不用手动写"更新TextView文本"这种重复代码，解决了老办法的"数据UI同步"痛点。
4. **兼容性好：**最低支持Android 5.0（API 21），覆盖99%以上的设备，不用操心老手机跑不了。

 比喻理解：XML写UI像"先画图纸（XML），再按图纸造房子（Kotlin逻辑）"，中途改图纸还要重新拆了再建；Compose像"直接用积木（代码）搭房子，想改哪块直接换，实时看到新样子"。

二、Compose的核心：可组合函数（最关键的 concept）

Compose的灵魂就是"可组合函数"，简单说就是：**用Kotlin函数来描述UI长啥样**。你把UI拆成一个个小部分（比如按钮、标题、列表项），每个部分都写成一个可组合函数，最后拼起来就是完整界面。

1. 可组合函数的3个核心特点

- 必须加@Composable注解（标记这是个UI函数）
- 函数名首字母大写（约定俗成，比如Button、Text）
- 没有返回值（不用写return，只负责描述UI）

2. 入门示例：写一个"我的第一个Compose界面"

咱们写一个包含"标题+按钮"的简单界面，代码直接复制到Android Studio就能运行：

```

1 // 1. 导入必要的包 (AS会自动提示导入, 不用手动记)
2 import androidx.compose.foundation.layout.Arrangement
3 import androidx.compose.foundation.layout.Column
4 import androidx.compose.foundation.layout.fillMaxSize
5 import androidx.compose.material3.Button
6 import androidx.compose.material3.Text
7 import androidx.compose.runtime.Composable
8 import androidx.compose.ui.Alignment
9 import androidx.compose.ui.Modifier
10
11 // 2. 主界面的可组合函数 (整个界面的入口)
12 @Composable
13 fun MyFirstComposeScreen() {
14     // Column: 垂直排列的容器 (像把东西从上到下摆)
15     Column(
16         modifier = Modifier.fillMaxSize(), // 占满整个屏幕
17         horizontalAlignment = Alignment.CenterHorizontally, // 子元素水平居中
18         verticalArrangement = Arrangement.Center // 子元素垂直居中
19     ) {
20         // 调用系统的Text可组合函数 (显示文字)
21         Text(text = "欢迎学习Compose! ")
22
23         // 调用系统的Button可组合函数 (按钮)
24         Button(onClick = {
25             // 按钮点击事件: 这里暂时先打印日志
26             println("按钮被点击啦! ")
27         }) {
28             // 按钮里的文字 (Button的子元素)
29             Text(text = "点我试试")
30         }
31     }
32 }
33
34 // 3. 预览函数 (不用运行App, 直接在AS里看效果)
35 @Composable
36 fun MyFirstComposeScreenPreview() {
37     MyFirstComposeScreen() // 直接调用主界面函数
38 }

```

3. 关键代码解释 (小白必看)

- **@Composable**: 告诉系统"这个函数是用来画UI的", 没有这个注解, 函数就不能描述UI。
- **Column**: 系统提供的"垂直布局容器", 就像一个竖排的盒子, 里面的元素从上到下排。类似的还有**Row** (水平布局, 左右排)。

- **Modifier:** "修饰器", 给UI元素加属性的工具, 比如这里的fillMaxSize()是"占满屏幕", 还能写Modifier.padding(16.dp) (加内边距)、Modifier.size(100.dp) (设置宽高)。
- **预览函数:** 加了@Composable的函数, 只要不依赖其他复杂数据, 就能写个预览函数实时看效果, 点击AS右侧的"Split"或"Design"就能看到界面。

三、搭骨架：Compose的布局系统

布局就是"把UI元素按想要的位置摆好", Compose的布局核心是「容器+子元素」, 比XML的LinearLayout、RelativeLayout简单多了, 常用的就3个基础布局:

1. 三大基础布局（必掌握）

布局类型	功能描述	简单代码示例
Column（垂直布局）	子元素垂直排列（从上到下）	Column { Text("第一行") Text("第二行") }
Row（水平布局）	子元素水平排列（从左到右）	Row { Text("左边") Text("右边") }
Box（单元素布局）	只能放一个子元素, 可控制子元素位置（比如居中、居右）, 适合做背景容器	Box(Modifier.fillMaxSize(), contentAlignment = Alignment.Center){ Text("屏幕正中间") }

2. 布局进阶：控制间距和大小

小白最常问的"怎么调间距", 用Modifier就能搞定, 看例子:

Code block

```
1  @Composable
2  fun LayoutDemo() {
3      Column(
4          modifier = Modifier
5              .fillMaxSize() // 占满屏幕
6              .padding(16.dp), // 整个Column距离屏幕边缘16dp
7          verticalArrangement = Arrangement.spacedBy(12.dp) // 子元素之间垂直间距
8      ) {
9          Text(
10             text = "带边距的文字",
11             modifier = Modifier
12                 .padding(8.dp) // 文字自身的内边距
13                 .background(color = Color.LightGray) // 加灰色背景, 方便看范围
```

```

14         )
15         Button(
16             onClick = {},
17             modifier = Modifier
18                 .fillMaxWidth() // 按钮占满父容器宽度
19                 .height(50.dp) // 按钮高度50dp
20         ) {
21             Text(text = "宽按钮")
22         }
23     }
24 }

```



单位注意：Compose里用dp（设备无关像素）做尺寸单位，不用px。写的时候直接写16.dp，AS会自动导入相关包。

四、穿衣服：Material Design 样式

Material Design（简称MD）是Google推荐的设计规范，包含了按钮、卡片、输入框等现成的"漂亮组件"，Compose已经把这些组件封装好了，咱们直接用就行，不用自己写样式。

1. 常用MD组件（小白优先掌握）

Code block

```

1  import androidx.compose.material3.Card
2  import androidx.compose.material3.OutlinedTextField
3  import androidx.compose.material3.Switch
4  import androidx.compose.material3.Text
5
6  @Composable
7  fun MaterialDesignDemo() {
8      Column(
9          modifier = Modifier.padding(16.dp),
10         verticalArrangement = Arrangement.spacedBy(16.dp)
11     ) {
12         // 1. 卡片组件（带阴影，常用作内容容器）
13         Card(modifier = Modifier.fillMaxWidth()) {
14             Text(text = "我在卡片里", modifier = Modifier.padding(16.dp))
15         }
16
17         // 2. 带边框的输入框
18         OutlinedTextField(
19             value = "", // 后面学状态管理会讲，暂时空着
20             onChange = {}, // 输入变化的回调
21             label = { Text(text = "请输入用户名") }

```

```

22         )
23
24         // 3. 开关组件
25         Row(verticalAlignment = Alignment.CenterVertically) {
26             Text(text = "是否记住密码")
27             Switch(checked = false, onCheckedChange = {})
28         }
29     }
30 }

```

这些组件自带MD风格，比如卡片的阴影、输入框的焦点效果，不用咱们写额外样式，大大减少工作量。

五、活起来：列表和动画

APP里的列表（比如微信聊天列表、新闻列表）和简单动画是高频需求，Compose实现起来比XML简单太多。

1. 列表：LazyColumn（懒加载列表）

为什么用LazyColumn？因为它"懒加载"——只加载屏幕上能看到的item，避免一次性加载几千条数据导致卡顿。

Code block

```

1  import androidx.compose.foundation.lazy.LazyColumn
2  import androidx.compose.foundation.lazy.items
3  import androidx.compose.material3.Text
4  import androidx.compose.runtime.Composable
5
6  @Composable
7  fun MyListView() {
8      // 模拟100条列表数据
9      val dataList = List(100) { "这是第${it+1}条列表数据" }
10
11     // 懒加载列表
12     LazyColumn(modifier = Modifier.fillMaxSize()) {
13         // 遍历数据生成item (items是系统提供的便捷方法)
14         items(dataList) { itemText ->
15             // 每个列表项的UI
16             Text(
17                 text = itemText,
18                 modifier = Modifier.padding(16.dp)
19             )
20         }
21     }

```

对比XML的RecyclerView，是不是省了Adapter、ViewHolder这些复杂代码？LazyColumn一句话就搞定列表！

2. 简单动画：让UI"动起来"

Compose把动画封装得很简单，比如做一个"点击按钮后文字变大"的动画：

Code block

```
1  import androidx.compose.animation.animateDpAsState
2  import androidx.compose.foundation.clickable
3  import androidx.compose.material3.Text
4  import androidx.compose.runtime.getValue
5  import androidx.compose.runtime.mutableStateOf
6  import androidx.compose.runtime.remember
7  import androidx.compose.runtime.setValue
8  import androidx.compose.ui.Modifier
9  import androidx.compose.ui.unit.dp
10 import androidx.compose.ui.unit.sp
11
12 @Composable
13 fun SimpleAnimationDemo() {
14     // 1. 记录动画状态 (是否放大, remember让状态在重组时保存)
15     var isEnlarged by remember { mutableStateOf(false) }
16
17     // 2. 动画效果: 文字大小从16sp变到24sp
18     val textSize by animateDpAsState(
19         targetValue = if (isEnlarged) 24.dp else 16.dp,
20         label = "textSizeAnimation"
21     )
22
23     // 3. 可点击的文字
24     Text(
25         text = "点击我放大",
26         fontSize = textSize,
27         modifier = Modifier.clickable {
28             isEnlarged = !isEnlarged // 点击切换状态
29         }
30     )
31 }
```

核心是**状态驱动动画**：我们只需要修改isEnlarged这个状态，动画会自动从"当前值"过渡到"目标值"，不用手动控制动画的开始和结束。

六、更高效：优化UI构建

Compose本身已经很高效了，但小白写代码时容易踩坑，记住以下2个优化技巧，就能让你的UI跑得更快：

1. 用remember保存临时数据

Compose的函数会频繁"重组"（比如数据变化时重新执行函数更新UI），如果每次重组都重新创建对象，会浪费资源。用remember可以把数据"记住"，避免重复创建。

Code block

```
1  @Composable
2  fun RememberDemo() {
3      // 错误写法：每次重组都会新建List
4      // val tempList = List(1000) { it }
5
6      // 正确写法：用remember保存，只创建一次
7      val tempList by remember {
8          mutableStateOf(List(1000) { it })
9      }
10 }
```

2. 拆分细粒度的可组合函数

不要把所有UI都写在一个函数里，拆成小函数，这样Compose只需要重组变化的小函数，而不是整个大函数。

Code block

```
1  // 不好的写法：所有UI挤在一起
2  @Composable
3  fun BadScreen() {
4      Column {
5          Text(text = "标题") // 不变的部分
6          Text(text = "当前时间: ${System.currentTimeMillis()}") // 每秒变化的部分
7      }
8  }
9
10 // 好的写法：拆分成小函数
11 @Composable
12 fun GoodScreen() {
13     Column {
14         TitleText() // 不变的部分，不会重组
15         TimeText() // 只重组这部分
16     }
17 }
```

```
18
19  @Composable
20  fun TitleText() {
21      Text(text = "标题")
22  }
23
24  @Composable
25  fun TimeText() {
26      Text(text = "当前时间: ${System.currentTimeMillis()}")
27  }
```

七、本章小结（小白必背）

1. Compose是用Kotlin写UI的工具，核心是"可组合函数"（加@Composable注解）。
2. 布局用Column（竖排）、Row（横排）、Box（单元素），配合Modifier控制样式和位置。
3. MD组件（Card、OutlinedTextField等）直接用，自带美观样式。
4. 列表用LazyColumn实现懒加载，动画靠"状态驱动"。
5. 优化技巧：用remember存数据，拆细粒度函数。

 课后作业：把今天的每个代码示例都复制到Android Studio里运行一遍，修改其中的参数（比如文字、颜色、间距），观察UI变化。动手实操是掌握Compose最快的方式！

（注：文档部分内容可能由 AI 生成）